

SIMATS ENGINEERING
MACHINE LEARNING (ITA0601)

STUDENT ACADEMICS

FACULTY: DR.R.KAVITHA

NAME: N HARSHAVARDHAN
REG NO: 192324189

QUESTION 1

You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

SOLUTION

The following pipeline can be used to satisfy the given constraints:

- Load the CSV file using Pandas and inspect the dataset using `df.info()`, `df.isnull().sum()`, and descriptive statistics.
- Handle missing values according to the data type. Numerical values can be replaced using suitable statistics such as median, while categorical values can be filled using the mode or an appropriate category.
- Use Pandas indexing and filtering to identify values outside medically valid ranges. For example, invalid ages, negative measurements, or impossible vital-sign values can be detected using Boolean conditions.
- Use NumPy functions such as `np.where()` to replace or flag invalid numerical values efficiently.
- Standardize categorical fields by removing extra spaces, converting text to a common case, and mapping different representations to one standard category.
- Use aggregation functions such as `mean()`, `median()`, `count()`, `min()`, and `max()` to verify the cleaned data and identify remaining anomalies.
- Finally, recheck missing values, valid ranges, duplicate records, and category values before saving the cleaned dataset. This preserves maximum data integrity because records are changed only when they violate defined constraints.

QUESTION 2

A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

SOLUTION

The following pipeline can be used to satisfy the given constraints:

- Create arrays with suitable data types such as float32 or int32 instead of unnecessarily using larger data types.
- Use NumPy functions such as `np.empty()`, `np.zeros()`, and `np.arange()` when appropriate instead of repeatedly creating temporary arrays.
- Process the dataset in chunks using slicing so that only a small portion of the data is held in memory at one time.
- Use in-place operations such as `+=`, `-=`, `*=`, and `/=` when safe. These reduce the creation of temporary arrays.
- Use random sampling to process only a representative subset when complete processing is not required. NumPy's random functions can generate efficient samples.
- Use aggregation functions such as `np.mean()`, `np.sum()`, `np.min()`, `np.max()`, and `np.var()` on slices or chunks and combine the results carefully.
- For debugging, monitor array shapes, data types, and memory usage. Check for unnecessary copies, oversized arrays, repeated computations, and inefficient loops. Profile execution time to locate bottlenecks and process data in smaller chunks if memory overflow occurs.

QUESTION 2

Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

SOLUTION

The following pipeline can be used to satisfy the given constraints:

- Inspect both DataFrames and check the customer ID column for missing values and duplicates using `isna()` and `duplicated()`.
- Standardize the customer ID data type and formatting before merging so that both DataFrames use the same key representation.
- Use Pandas `merge()` with the appropriate join type. An outer join can be used when preservation of all valid records from both datasets is required.
- After merging, use grouping and counting to verify that each customer ID occurs only as expected. For example, `groupby('customer_id').size()` can identify unexpected duplicates.
- Use filtering to locate unmatched records and missing values in fields originating from either DataFrame.
- Reset or set the index appropriately after the merge using `reset_index()` or `set_index()`, and sort the final DataFrame by customer ID for consistent organization.
- Debug mismatches by checking data types, leading/trailing spaces, formatting differences, missing IDs, and duplicate keys. Compare record counts before and after merging to ensure that valid records were not unintentionally lost.

QUESTION 2

You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

SOLUTION

The following pipeline can be used to satisfy the given constraints:

- Load the numerical data using Pandas and inspect its data type, missing values, and invalid entries before calculating statistics.
- Use Pandas aggregation functions such as `mean()`, `median()`, and `var()` or equivalent NumPy functions `np.mean()`, `np.median()`, and `np.var()`.
- Use filtering and Boolean indexing to remove or exclude irrelevant records before performing calculations. This prevents unwanted values from affecting the results.
- Avoid redundant computations by calculating the required statistics once and reusing the cleaned or filtered data.
- For numerical accuracy, use an appropriate numerical data type and avoid unnecessary conversions that may introduce precision loss. NumPy can perform calculations efficiently on arrays.
- Use aggregation across the required axis and avoid Python-level loops where vectorized NumPy/Pandas operations are available.
- Debug incorrect outputs by checking the filtered dataset, data types, missing values, sample size, and whether Pandas and NumPy are using different default variance settings. Compare results on a small manually verified subset before processing the complete dataset.

QUESTION 2

Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

SOLUTION

The following pipeline can be used to satisfy the given constraints:

- Identify the available files and load them using suitable Pandas functions such as `pd.read_csv()`. For large files, use options such as `chunksize` when necessary to reduce memory usage.
- Inspect the structure, column names, data types, and missing values of each DataFrame. Standardize column names and formats before combining datasets.
- Combine compatible DataFrames using `concat()` or `merge()` depending on whether the files contain similar records or related keys.
- Use filtering with Boolean conditions to select records according to user-defined requirements.
- Use `groupby()` with aggregation functions such as `sum()`, `mean()`, `count()`, and `median()` to produce meaningful summaries.
- Use `sort_values()` and suitable indexing to arrange the processed results correctly before visualization.
- Generate meaningful plots such as bar charts, line charts, histograms, or scatter plots based on the analysis objective. Labels, titles, and suitable axes should make the output understandable.
- For debugging, catch file-reading errors, inspect corrupted files separately, verify column names and data types, check for inconsistent date or numeric formats, and compare record counts before and after each processing stage.